

INDUZIONE STRUTTURALE

4) $E \rightarrow n \mid E+E \mid E * E \rightsquigarrow$ Def exp (insieme delle espressioni aritmetiche)
 $n \in \mathbb{N}$

$$\begin{cases} \text{Base: } n \in \text{Exp} \\ \text{P.ind: } e_1, e_2 \in \text{Exp} \text{ allora } e_1 + e_2, e_1 * e_2 \in \text{Exp} \end{cases}$$

Es. $3 + 5 * 2 \in \text{Exp}$

$$\text{val} : \text{Exp} \rightarrow \mathbb{N} \quad \text{op} : \text{Exp} \rightarrow \mathbb{N}$$

conta il n° di valori conta il n° di op.

$$\begin{cases} \text{val}(n) = 1 \\ \text{val}(e_1 + e_2) = \text{val}(e_1 * e_2) = \text{val}(e_1) + \text{val}(e_2) \end{cases}$$

$$\begin{cases} \text{op}(n) = 0 \\ \text{op}(e_1 + e_2) = \text{op}(e_1 * e_2) = \text{op}(e_1) + \text{op}(e_2) + 1 \end{cases}$$

$$\text{val}(\underbrace{3+5}_{e_2} * \underbrace{2}_{e_1}) = \text{val}(\underbrace{3+5}_{e_1} + \underbrace{2}_{e_2}) = \underbrace{\text{val}(3)}_1 + \underbrace{\text{val}(5)}_1 + \underbrace{\text{val}(2)}_1 = 3$$

$$\text{op}(\underbrace{3+5}_{e_1} * \underbrace{2}_{e_2}) = \text{op}(\underbrace{3+5}_{e_1} + \underbrace{2}_{e_2}) + 1 = \underbrace{\text{op}(3)}_0 + \underbrace{\text{op}(5)}_0 + 1 + \underbrace{\text{op}(2)}_0 + 1 = 2$$

Dim. Π : $\forall e \in \text{Exp} \quad \text{val}(e) = \text{op}(e) + 1$. Dim. induzione strutturale

Base: verifichiamo Π sugli element: esplicitamente
inseriti in Exp : $n \in \mathbb{N} \quad e = n$

$$\text{val}(e) = \text{val}(n) = 1 = 1 + 0 = 1 + \text{op}(n) = 1 + \text{op}(e) \quad \checkmark$$

P.ind. : $e = e_1 + e_2$ (analogo per $e = e_1 * e_2$)

Hp.ind. $val(e) = qp(e_1) + 1$

$val(e_2) = qp(e_2) + 1$

Dobbiamo dim. : $val(e) = qp(e) + 1$

$$val(e) \stackrel{\text{ipotesi}}{=} val(e_1 + e_2) \stackrel{\text{def. d. val}}{=} val(e_1) + val(e_2)$$

$$\stackrel{\text{Hp.ind.}}{=} qp(e_1) + 1 + qp(e_2) + 1$$

$$\stackrel{\text{def. d. qp}}{=} qp(e_1 + e_2) + 1 \stackrel{\text{ipotesi}}{=} qp(e) + 1 \quad \checkmark$$

Semantica → Attribuire significato ai simboli

↳ comportamento I/O (denotazionale) → (descrive funzionalità: programmi come funzioni)

↳ fornire gli algoritmi di esecuzione (operazionale) → (descrive il comportamento del progr. come seq. di trasformazioni di stati)

↳ proprietà invariante dei programmi (assiomatica) → (descrive il progr. attraverso proprietà che soddisfa)

Esempio

P: $z := z;$

semantica di P?

$y := z;$

Semantica denotazionale

$y := y + 1;$

Modello matematico che descrive il programma

$z := y;$

come una funzione su stati

stato: descrive il valore delle variabili

$var \rightarrow val$

D: $Prog \rightarrow (stato \rightarrow stato)$ /* funzione che prende uno stato e restituisce uno stato x/

denotazione associata al programma della semantica

[0:28:00]

$$D[P] (\underbrace{z \mapsto 1, y \mapsto 1}_{\text{stato iniziale}}) = D[z := z; y := z; y := y + 1; z := y] (z \mapsto 1, y \mapsto 1)$$

$$= D[z := y] \circ D[y := y + 1] \circ D[y := z] \circ \underline{D[z := z]} (z \mapsto 1, y \mapsto 1)$$

$$= D[z := y] \circ D[y := y + 1] \circ \underline{D[y := z]} (z \mapsto 2, y \mapsto 1)$$

$$= D[z := y] \circ \underline{D[y := y + 1]} (z \mapsto 2, y \mapsto 2)$$

$$= \underline{D[z := y]} (z \mapsto 2, y \mapsto 3) = (z \mapsto 3, y \mapsto 3)$$

usata per

no dim. proprietà (correttezza)

→ fornisce uno strumento formale per ragionare sulle funzionalità di programmi:

⇒ non è utile a chi usa il PL

Semantica assiomatica

↳ descrive relazioni tra proprietà di stati:

[0:40:00]

⇒ usata per dimostrare proprietà

Semantica operazionale

↳ descrive il significato dei programmi "eseguendo" i suoi comandi sulla macchina (astratta)

stato/memoria $\sigma: \text{Var}^n \rightarrow \text{Val}^n$ /* associa valore alle variabili */

Variabili $\langle x, y, z \rangle \mapsto \langle 2, 5, 3 \rangle$

$\Downarrow$
 $\langle x \mapsto 2, y \mapsto 5, z \mapsto 3 \rangle$

sistema di transizione $\langle \Sigma, \rightarrow \rangle$
 ← relazione di transizione
 ← stati

Σ : stato: $(\text{Prog} \times \text{mem}) \cup \text{mem}$
 ← stati finali

quello che dobbiamo eseguire
 memoria su cui eseguiamo il programma

Es. $\langle P, (z \mapsto 1, y \mapsto 1) \rangle$
 ← prog. ← mem. iniziale
 ← stato iniziale

$= \langle z := 2; y := z; y := y + 1; z := y, (z \mapsto 1, y \mapsto 1) \rangle$
 $= \langle y := z; y := y + 1; z := y, (z \mapsto 2, y \mapsto 1) \rangle$
 $= \dots$
 $= (z \mapsto 3, y \mapsto 3)$

traccia di esecuzione e la semantica di P

[0:51:21]

COMPOSIZIONALITÀ della semantica

↳ il significato di ogni programma è funzione del significato dei suoi componenti immediati

$C = C_1 ; C_2$
 $\downarrow \quad \downarrow$
 $\text{sem}(C_1) \text{ o } \text{sem}(C_2) \rightarrow \text{sem}(C)$

EQUIVALENZA di programmi (mediante semantica)

$P_1 = P_2$ (uguaglianza sintattica)

$P_1 \equiv P_2$ (equivalenza $\neq$ uguaglianza semantica)

P_1 insertion sort	$P_1 \neq P_2$
P_2 bubble sort	$D[P_1] = D[P_2] \Rightarrow P_1 \equiv_D P_2$
	$[P_1] \neq [P_2] \Rightarrow P_1 \not\equiv_{[P]} P_2$ sem. φ .

$\Rightarrow$ Equivalenza dipende dalla scelta della semantica

CATEGORIE SINTATTICHE

$\hookrightarrow$ manipolare valori (espressioni)

$\hookrightarrow$ trasformazioni di stati (comandi)

$\hookrightarrow$ creazione (e manipolazione) di legami (dichiarazioni)

stato (no memoria) $\Rightarrow$ descrive effetti su: identificatori

ambiente $\Rightarrow$ insieme di legami tra identificatori e oggetti denotati

Espressioni $\Rightarrow$ Denotano valori:

$\Rightarrow$ le espressioni vengono valutate per restituire valori

$\Rightarrow$ espressioni sintatticamente diverse possono denotare lo stesso valore

$1+5 \quad \} \quad 3*2$
 $\quad \downarrow$
espressioni equivalenti

Due espressioni sono equivalenti se vengono valutate nello stesso valore a partire da qualunque stato/memoria iniziale

Comandi $\Rightarrow$ Denotano richieste di modifica della memoria

rappresentano funzioni da memoria a memoria

$\Rightarrow$ i comandi vengono eseguiti per attuare la corrispondente

richiesta di trasformazione della memoria

$\downarrow$
irreversibili

=> due comandi sono equivalenti se per ogni stato iniziale in input producono lo stesso stato (memoria) in output

ma equivalenza $\equiv$ uguaglianza della funzione calcolata

Dichiarazioni ma Denotano richieste di creazione o modifica di legami
ovvero di ambienti:

↳ descrivono legami tra identificatori e oggetti denotati

↳ le dichiarazioni vengono elaborate per attuare le richieste
di creazione/modifica degli ambienti

↳ due dichiarazioni sono equivalenti se producono lo stesso
ambiente (la stessa memoria) a partire da qualunque stato iniziale